

How Do Different Cache Policies React When Workload Changes Over Time?

Erik B. Ebbestad

Dept. of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

erebb5820@uit.no

Ebbisso@github

Frode Eggenfellner

Dept. of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

fregg8888@uit.no

Frodhesten@github

Martin Rasmussen

Dept. of Computer Science

UiT The Arctic University of Norway

Tromsø, Norway

maras7515@uit.no

Martinstudererinformatick@github

I. GROUP DESCRIPTION

Our group consists of three students from the Department of Computer Science at UiT The Arctic University of Norway: Erik B. Ebbestad, Frode Eggenfellner, and Martin Rasmussen.

The group will investigate cache replacement policies and how their performance is affected by changes in workload behaviour over time. The project combines computer architecture, performance measurement, and controlled systems experiments.

II. INTRODUCTION

Modern computer systems rely heavily on caches to reduce the latency between the processor and slower levels of the memory hierarchy. Since cache capacity is limited, a cache must decide which data to keep and which data to evict when new data needs to be inserted. This decision is controlled by a cache replacement policy.

Common replacement policies such as Least Recently Used (LRU), First-In First-Out (FIFO), and Random make different assumptions about how programs access memory. For example, LRU benefits from temporal locality because recently accessed data is expected to be accessed again. However, the characteristics of a workload do not necessarily remain constant throughout its execution.

A program may contain several execution phases with substantially different memory-access patterns. One phase may repeatedly access a small working set, while another phase may sequentially process a large amount of data. As a result, a cache replacement policy that performs well during one phase may perform poorly during another.

This project will investigate this behaviour through controlled experiments. The main objective is to determine how different cache replacement policies respond when the characteristics of the workload change over time.

A. Motivation and Goal

The motivation for this project is that cache replacement policies are often evaluated using workloads where the overall memory-access behaviour is treated as relatively stable. In real applications, however, memory-access patterns can change during execution. Understanding the effect of such changes

is important when evaluating cache policies because a policy may not immediately adapt to a new workload phase.

The goal of the project is therefore to experimentally compare several cache replacement policies under workloads containing changes in memory-access behaviour. In particular, we want to measure both the performance of each policy before and after a workload transition and how quickly the policy recovers from the transition.

The planned experiments will focus on cache hit rate, miss rate, and potentially additional performance metrics such as average memory access time (AMAT). The experiments will be designed so that cache size, associativity, block size, and workload length can be controlled independently.

B. Research Question

The main research question is:

How does the performance of different cache replacement policies change when the workload transitions between different memory-access patterns?

To investigate this question, we define the following sub-questions:

- **RQ1:** How do different cache replacement policies perform before and after a change in workload locality?
- **RQ2:** How quickly do different replacement policies adapt to a new workload phase?
- **RQ3:** How does the frequency and magnitude of workload changes affect the relative performance of different cache policies?

We expect policies that rely strongly on recent access history to be affected by workload transitions because information from the previous phase may remain in the cache after the transition. We will test this hypothesis experimentally rather than assuming that one policy will always outperform the others.

III. RELATED WORK

Cache replacement policies have been studied extensively because eviction decisions directly dictate memory hierarchy performance. Classical non-adaptive heuristics, such as Least Recently Used (LRU), First-In First-Out (FIFO), and Least

Frequently Used (LFU), rely on fixed static assumptions regarding memory locality. While LRU effectively captures short-term temporal locality, it is vulnerable to cache pollution during streaming phases or rapid working-set expansions. Conversely, LFU captures long-term object popularity but suffers from frequency pollution, failing to evict items that become inactive after a workload phase transition.

To mitigate these limitations, modern systems employ adaptive algorithms that dynamically respond to non-stationary workloads. The Adaptive Replacement Cache (ARC) by Megiddo and Modha [1] established a framework for dynamically tuning the balance between recency and frequency via ghost caches. In CPU architectures, Dynamic Insertion Policies (DIP) by Qureshi et al. [2] introduced set dueling to adaptively choose block insertion depth based on access patterns, preventing thrashing during scan-heavy execution phases.

Recent research has focused heavily on lightweight policies capable of surviving sudden shifts in access distribution, phase transitions, and high "one-hit-wonder" ratios:

- **S3-FIFO and SIEVE:** Yang et al. [4] and Zhang et al. [5] demonstrated that traditional LRU strategies can be outperformed by simplified, FIFO-based queue structures combined with lazy promotion and quick demotion mechanisms. SIEVE utilizes a single FIFO queue with a moving pointer to rapidly evict non-reaccessed items upon arrival, offering extreme adaptivity under heavy workload fluctuations without expensive global list updates.
- **Machine Learning and Reinforcement Learning-Based Eviction:** Recent studies by Zhou et al. [6] apply deep reinforcement learning (RL) to dynamically model the relationship between shifting workload distributions and optimal eviction choices. These policies continuously adapt eviction parameters to non-stationary access patterns, though at the expense of higher runtime overhead compared to classical heuristics.
- **Analytical Modeling of Phase Shifts:** Theoretical foundations, such as the Shot-Noise Model (SNM) frameworks by Traverso et al. [3], analyze how hit ratios decay and recover when content popularity undergoes abrupt temporal transitions.

While existing literature frequently benchmarks the overall average hit rates of adaptive vs. static policies over complete execution traces, fewer empirical studies isolate the *transient recovery behavior* immediately following a drastic locality transition. Our work explicitly focuses on this gap by measuring recovery latency, transient miss spikes, and phase-adaptation dynamics across classical and lightweight modern replacement policies under controlled workload shifts.

IV. SYSTEM

The system under study will be a simplified cache and memory hierarchy. The cache will receive a sequence of memory accesses generated by controlled workloads. For each access, the cache will determine whether the requested block is already present. On a cache miss, a replacement policy will

determine which existing cache block should be evicted when the cache is full.

The initial experiment will compare several conventional replacement policies:

- **LRU:** evicts the cache line that has not been accessed for the longest time.
- **FIFO:** evicts the cache line that has been present in the cache for the longest time.
- **Random:** selects a cache line for eviction randomly.
- **LFU:** evicts the cache line with the lowest access frequency.

Depending on the complexity of the implementation and the results of the initial experiments, an adaptive replacement policy may also be included.

The workloads will consist of multiple phases with different memory-access patterns. A simplified example is:

$$W = W_A \rightarrow W_B \rightarrow W_A \quad (1)$$

where W_A and W_B represent workloads with different locality characteristics. For example, W_A may repeatedly access a small working set, while W_B may access a larger range of addresses sequentially.

This structure allows us to measure the performance of a cache policy before, during, and after a workload transition. We will also vary the duration of individual phases and the difference between their access patterns.

The main dependent metrics will be cache hit rate and cache miss rate. Additional metrics such as AMAT may be included if the simulator provides sufficient timing information.

V. EARLY ENVIRONMENT AND SCOPE

The initial implementation is planned as a controlled cache simulator rather than a measurement of a complete physical CPU cache. This approach allows us to isolate the replacement policy from other hardware effects and precisely control the workload.

The simulator will initially be implemented in C or another suitable programming language. It will model a configurable cache with parameters such as:

- cache capacity,
- cache block size,
- cache associativity,
- replacement policy,
- number of memory accesses,
- workload phase length, and
- memory-access pattern.

The experiments will initially use synthetic workloads because they provide direct control over locality and workload transitions. Later in the project, real application traces may be considered if they can be obtained and integrated without making the experimental setup unnecessarily complex.

To improve reproducibility, each experiment will use fixed configuration parameters and, where randomness is involved, controlled random seeds. Multiple repetitions will be performed for experiments involving randomized behaviour.

The initial scope is deliberately limited to cache replacement policies. Other cache mechanisms, such as prefetching, branch prediction, compiler optimizations, and coherence protocols, will not be investigated in the first stage because they could introduce additional variables that make it harder to isolate the effect of workload changes.

VI. EARLY TIMELINE

The project will be developed in several stages throughout the semester. We will first refine the research questions and investigate existing work on cache replacement policies, locality, and changing workload behaviour. This will help us define the experimental methodology and determine which cache policies and workload characteristics should be included.

The next step will be to design and implement the cache simulator. The simulator will provide a controlled environment where the cache configuration, replacement policy, and memory-access workload can be changed independently. The different replacement policies will then be implemented and tested to verify that the simulator behaves as expected.

After the simulator has been validated, we will create synthetic workloads with different memory-access patterns. These workloads will be constructed from multiple phases so that the memory-access behaviour changes during execution. Initial pilot experiments will be used to determine suitable cache configurations, workload sizes, and phase lengths.

The main experiments will then be performed using the selected configurations. We will measure cache hit rate and miss rate before and after workload transitions, and investigate how the different replacement policies respond to these changes. Where appropriate, the experiments will also measure the time or number of accesses required for a policy to recover after a workload transition.

Finally, the collected results will be analysed and compared across the different policies and workload configurations. The results will be used to answer the research questions and evaluate the hypotheses. Based on the findings, we will discuss the limitations of the experimental setup and whether the results can be generalized beyond the simulated system.

VII. DISCUSSION

Several risks and limitations have been identified at this early stage.

First, designing workloads that represent meaningful changes in behaviour is challenging. If the difference between workload phases is too small, the experiment may not reveal significant differences between policies. If the difference is too large, the experiment may become unrealistic. We will therefore begin with simple, controlled patterns and gradually increase their complexity.

Second, a cache simulator does not necessarily represent the behaviour of a real processor cache. Real systems contain multiple cache levels, hardware prefetchers, complex replacement mechanisms, and other effects. Our initial goal is not to reproduce a particular CPU exactly, but to isolate the effect of

replacement policies under controlled conditions. The limitations of this approach will be discussed when interpreting the results.

Third, the number of parameters that can be varied is large. Cache capacity, associativity, block size, workload size, phase duration, and access patterns can all affect the results. Varying too many parameters could make the experiment unnecessarily large. We will therefore use a small number of controlled configurations and change one major factor at a time where possible.

Another risk is that overall hit rate may hide the behaviour immediately following a workload transition. To address this, measurements will be collected per workload phase, and where appropriate over smaller intervals around the transition. This will allow us to distinguish long-term cache performance from short-term adaptation behaviour.

Finally, the project may initially focus too narrowly on traditional policies. If the basic experiments are completed early enough, an adaptive replacement policy will be added to provide a comparison between fixed and adaptive strategies.

Overall, the project is intended to provide a controlled experimental study of how cache replacement policies behave when the assumptions about workload locality change during execution. The results should help clarify whether the ranking of replacement policies remains stable across workload phases, and whether some policies are better suited to workloads with frequent changes in memory-access behaviour.

REFERENCES

- [1] algorithm - C - How to implement Set data structure? - Stack Overflow. Retrieved from <https://stackoverflow.com/questions/2630738/c-how-to-implement-set-data-structure>
- [2] Patino, A. (2024, November 9). Cache replacement policies explained. Retrieved September 7, 2026, from Aerospike website: <https://aerospike.com/blog/cache-replacement-policies/>
- [3] Gill, B., Modha, D. (n.d.). SARC: Sequential Prefetching in Adaptive Replacement Cache. Retrieved from https://www.usenix.org/legacy/event/usenix05/tech/general/full_papers/gill/gill.pdf
- [4] Kandaanusha. (2025, October 28). Cache Eviction Policies. Retrieved September 7, 2026, from Medium website: <https://medium.com/@kandaanusha/cache-eviction-policies-70b9f3cfd90b>